Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-07-29

Contents

Executive Summary — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 5

1. Solution Overview — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 6

- What this solution is . . . 6
- What Event ID 5719 actually means . . . 6
- The root-cause tree (what Discovery checks, and why) . . . 6
- The IPv6 finding — keep it enabled, fix invalid values only . . . 7
- How you run it (local, or at scale via Proactive Remediations) . . . 7
- Named objects you create (and reuse) . . . 8
- Benefits . . . 8
- Limitations (be honest with buyers) . . . 8
- The standard diagnostics model . . . 8

2. Requirements — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 9

- R1. Domain-joined Windows devices . . . 9
- R2. Local administrator / SYSTEM execution context . . . 9
- R3. Windows PowerShell 5.1+ with in-box modules . . . 9
- R4. In-box domain tooling present (`nltest`, `w32tm`) . . . 10
- R5. Network path to a domain controller (for repair) . . . 10
- R6. Correct internal DNS on the device NIC . . . 10
- R7. Intune Proactive Remediations available (for at-scale runs) . . . 10
- R8. A place to collect output and escalation bundles . . . 10
- R9. Pilot device and pilot group . . . 11

Global Requirements — Intune Solutions (category-level) 11

- G1. Microsoft Entra ID tenant . . . 11
- G2. Microsoft Intune subscription . . . 11
- G3. Administrative roles . . . 11
- G4. Device enrollment in Intune . . . 11
- G5. Pilot security group . . . 11
- G6. Supported Windows baseline . . . 12
- G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

- Phase 1 — Assess (get the scripts onto the device) . . . 12
- Phase 2 — Diagnose (collect every signal, read-only) . . . 13
- Phase 3 — Repair (preview first, then apply) . . . 13

Phase 4 — Validate . . . 14
Phase 5 — Deploy at scale via Intune Proactive Remediations (and Prevent) . . . 14
5. Testing & Validation — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 15
Test 1 — Discovery collects every signal . . . 15
Test 2 — Validation gate and exit code . . . 16
Test 3 — Secure-channel repair (preview then apply) . . . 16
Test 4 — Time re-sync . . . 16
Test 5 — DNS DC-locator resolution . . . 16
Test 6 — IPv6 recommended configuration . . . 16
Test 7 — DC port reachability . . . 16
Test 8 — Report and escalation bundle . . . 17
Test 9 — Proactive Remediations at scale . . . 17
Test 10 — No enrollment/join change (negative test) . . . 17
Acceptance gate . . . 17
6. Troubleshooting — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 17
Symptom: Event ID 5719 keeps appearing, but Validation says HEALTHY . . . 17
Symptom: `Test-ComputerSecureChannel` returns False (trust broken) . . . 18
Symptom: DNS SRV records do not resolve (DC Locator fails) . . . 18
Symptom: Clock skew — time is more than 5 minutes off . . . 18
Symptom: A DC port shows "blocked" (88/135/389/445) . . . 19
Symptom: IPv6 value flagged INVALID / unsupported . . . 19
Symptom: Remediation says "Preview only" and changed nothing . . . 19
Symptom: Works on one device, not another . . . 19
Symptom: Proactive Remediations shows "Issues detected" but never remediates . . . 19
Escalation checklist . . . 20
7. Best Practices — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 20
Diagnose before you touch anything . . . 20
Repair safely (preview-first) . . . 20
Keep IPv6 in a supported configuration (never blanket-disable) . . . 20
Prevent recurrence (the boot-time race and drift) . . . 21
Operate at scale with Proactive Remediations . . . 21
Keep evidence and change control . . . 21
8. Deliverables Summary — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit 22
Documentation (this guide) . . . 22

PowerShell scripts (endpoint) — `NETLOGONDX-Scripts.zip` 22
Hero image 23
How it's delivered 23
Appendix A — PowerShell Scripts 24

Executive Summary — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Domain-joined Windows devices intermittently fail to log on, drop mapped drives, or lose Group Policy — and the System log fills with **Netlogon Event ID 5719: "There are currently no logon servers available to service the logon request."** Behind that one message hides a small set of very different root causes: a **broken secure channel** (the machine account/trust password out of sync), **DNS** that can no longer locate a domain controller, **clock skew** that breaks Kerberos, **blocked network ports** to the DCs, **no reachable DC** on the subnet at boot, a **NIC/power-management race** that drops the link before Netlogon starts — and, surprisingly often, an **invalidly disabled IPv6 stack**. This toolkit finds which one it is and fixes it, on one device or ten thousand.

A diagnostics toolkit, not a config baseline. This package is **read-heavy endpoint PowerShell** that runs **locally on the affected Windows device** or **at scale via Intune Proactive Remediations**. It follows the repeatable diagnostics arc — **Assess** → **Diagnose** → **Repair** → **Validate** → **Prevent** — and resolves the two recurring **availability findings** it is built around: **Event ID 5719 (Netlogon)** and **"set IPv6 to a recommended configuration."**

The five scripts are deliberately safe by default:

- `NETLOGONDX-Discovery.ps1` — collects the whole picture in one pass: recent 5719 and related

secure-channel events, `Test-ComputerSecureChannel`, `nltest /sc_query` and `/dsgetdc`, DNS SRV resolution of the DC locator records, `w32tm` time state, DC port reachability (88/135/389/445), and the IPv6 `DisabledComponents` registry value. **Read-only.**

- `NETLOGONDX-Validation.ps1` — a hard pass/fail gate (and a ready-made Proactive Remediations

detection script): secure channel healthy, time in sync, DNS resolves DCs, IPv6 not invalidly disabled. **Read-only.**

- `NETLOGONDX-Report.ps1` — a readable + CSV health summary for fleet roll-up. **Read-only.**
- `NETLOGONDX-Remediation.ps1` — repairs the secure channel (`Test-ComputerSecureChannel -Repair`,

falling back to `nltest /sc_reset`), resyncs time, and **corrects an invalid IPv6** `DisabledComponents` **back to the recommended value**. **Defaults to** `-WhatIf` — it previews every change and modifies nothing until you pass `-WhatIf:$false`.

- `NETLOGONDX-Collect.ps1` — bundles the logs and command output into one ZIP for escalation.

Read-only.

On IPv6, the guidance is explicit and correct: IPv6 should stay **ENABLED** (the dual-stack default). The toolkit **never disables IPv6** — it identifies **invalid** `DisabledComponents` values (such as `0xffffffff`) and corrects them back to a **supported** value (`0x0` default, or `0x20`/`0x11` only with justification). Disabling IPv6 outright breaks **DirectAccess, Remote Assistance, and HomeGroup** and is **not covered by Microsoft's testing**.

The package delivers the complete, field-tested method — written so a single administrator can follow it end to end — with exact steps to run the toolkit locally or deploy it via Proactive Remediations for at-scale detection and remediation, plus testing and troubleshooting guides with clear pass/fail gates.

Outcome: the real cause of Event ID 5719 identified and repaired on any domain-joined Windows device, IPv6 restored to a supported configuration, and a repeatable health check you can run on the whole fleet — without a consulting engagement and without touching how devices are enrolled.

1. Solution Overview — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

What this solution is

This solution is a **diagnostics toolkit** — five endpoint PowerShell scripts plus a complete method — that pinpoints and repairs the root cause of **Netlogon Event ID 5719** and a **broken domain secure channel** on domain-joined Windows devices, and restores **IPv6 to a recommended configuration**. The scripts run **locally on the affected device** (elevated) or **at scale under SYSTEM via Microsoft Intune Proactive Remediations**. They are **read-heavy by design**: four of the five change nothing; only the remediation script makes changes, and it **defaults to** `-WhatIf` (preview) until you explicitly apply.

> ***Scope, stated plainly.*** *This is an* ***endpoint diagnostics toolkit*** *for the *client* side of the domain trust — the sibling of our endpoint performance/diagnostics toolkits. It is* ***not*** *a configuration baseline,* ***not*** *a domain-controller build guide, and* ***not*** *an Active Directory health tool for the DCs themselves. It resolves two specific availability findings on Windows endpoints:* ***Event ID 5719 (Netlogon)*** *and* ***"set IPv6 to a recommended configuration."***

What Event ID 5719 actually means

NETLOGON logs **Event ID 5719** — *"This computer was not able to set up a secure session with a domain controller… There are currently no logon servers available to service the logon request"* — whenever the device cannot establish the authenticated **secure channel** it needs to a domain controller (DC). The message is a **symptom with several distinct causes**. This toolkit walks the full root-cause tree rather than guessing.

The root-cause tree (what Discovery checks, and why)

1. **DNS / DC locator.** The device finds DCs by resolving **SRV locator records** —

`_ldap._tcp.dc._msdcs.<domain>` — against **internal AD DNS**. If the NIC points at a public resolver (8.8.8.8) or the wrong DNS server, the DC Locator fails and 5719 follows. *Discovery resolves the SRV record and reports the client's DNS servers.*

1. **Secure channel / trust password.** Each domain member holds a **machine account password** that

must match the copy in AD. It falls out of sync when a device is **restored from a snapshot/image**, **cloned without sysprep**, or **left off the domain longer than the machine-password age**. Then `Test-ComputerSecureChannel` returns **False**. *Discovery runs that test and `nltest /sc_query`.*

1. **Time skew (Kerberos).** Kerberos rejects tickets when the client clock is more than **5 minutes**

off the DC. A drifted clock (dead CMOS battery, wrong time source) breaks authentication and logs

1. *Discovery reads `w32tm` state and measures the offset against a located DC.*

2. **Network path & firewall.** The secure channel needs the DC reachable on **88 (Kerberos)**,

135 (RPC endpoint mapper), **389 (LDAP)**, and **445 (SMB)**. A firewall/VLAN/segmentation change silently blocks them. *Discovery TCP-probes all four against the located DC.*

1. **DC reachability.** The subnet may map to an **AD site with no DC**, or a **VPN/DirectAccess**

tunnel may not be up at boot, so no DC answers the locator. *Discovery reports the located DC (or none) and port results.*

1. **NIC / power-management race at boot.** A classic **transient** 5719: the network adapter isn't up

when Netlogon starts, often because **"Allow the computer to turn off this device to save power"** is enabled on the NIC, or **"Always wait for the network at computer startup and logon"** is not set. *Best Practices covers preventing this; Discovery captures the boot-time 5719 pattern.*

1. **IPv6 invalidly disabled.** Windows domain networking is built and tested as **dual-stack**. An

invalid `DisabledComponents` value (see below) can break name resolution and DC location and is a common, self-inflicted cause. *Discovery reads and classifies the value.*

The IPv6 finding — keep it enabled, fix invalid values only

The registry value `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters\DisabledComponents` (DWORD) tunes IPv6. This toolkit takes the **Microsoft-supported** position:

- **IPv6 must remain ENABLED.** The recommended value is `0x0` (or the value simply **absent**) —

full **dual-stack**. Do **not** disable IPv6.

- **Supported, only with justification:** `0x20` (prefer IPv4 while IPv6 stays enabled) and

`0x11` (disable IPv6 on non-tunnel + tunnel interfaces). Use these deliberately, not by accident.

- **Invalid / unsupported:** values such as `0xffffffff` (and the blanket-disable `0xff`) set

bits Microsoft **does not test**. They are the ones to **correct back to** `0x0`.

- **Why not just disable it:** disabling IPv6 breaks **DirectAccess**, **Remote Assistance**, and

HomeGroup, and Microsoft **does not test** Windows with IPv6 disabled — so it is unsupported.

The Remediation script only ever **corrects invalid values to** `0x0`; it leaves supported values untouched and **never disables IPv6**.

How you run it (local, or at scale via Proactive Remediations)

- **On one device (triage):** run `NETLOGONDX-Discovery.ps1` elevated to see the whole picture, then

`NETLOGONDX-Remediation.ps1` (preview first, then `-WhatIf:$false`) to repair, and `NETLOGONDX-Validation.ps1` to confirm.

- **Across the fleet (at scale):** package `NETLOGONDX-Validation.ps1` **as the DETECTION script** and

`NETLOGONDX-Remediation.ps1` **as the REMEDIATION script** in an Intune **Proactive Remediations** script package, assigned to a pilot group first. Detection exits `1` on any unhealthy signal, which triggers

remediation; Report/Collect gather evidence. All scripts use **in-box Windows commands only** — no modules to install on endpoints.

Named objects you create (and reuse)

- `C:\ProgramData\NETLOGONDX` — the on-device output folder every script writes to (Discovery JSON,

Report CSV/TXT, Collect ZIP). Referenced by name throughout.

- `NETLOGONDX – Secure-Channel & Netlogon Health` — the Proactive Remediations script package

(detection = Validation, remediation = Remediation).

- `NETLOGONDX-Pilot` — the Entra security group you assign the package to first.

Benefits

- **One pass, real answer.** Discovery replaces a scavenger hunt across Event Viewer, `nltest`,

`w32tm`, DNS, and the registry with a single, ordered report.

- **Safe by default.** Four read-only scripts; the only script that changes anything previews first.

- **Local or at scale.** The same scripts triage one device or run fleet-wide via Proactive

Remediations.

- **Correct IPv6 posture.** Restores the supported dual-stack default without the unsupported

"just disable IPv6" mistake.

- **Escalation-ready.** One command bundles everything a support case needs.

Limitations (be honest with buyers)

- **Domain-joined Windows endpoints only.** The scripts short-circuit on non-domain devices. They do

not diagnose the domain controllers themselves (use AD/DC health tooling for that).

- **Repairs need connectivity to a DC.** `Test-ComputerSecureChannel -Repair` / `nltest /sc_reset`

require the device to reach a DC and (in some cases) domain credentials; a device with no DC path can be diagnosed but not repaired until the path is restored.

- **IPv6 changes need a restart** to take effect.

- **Values can evolve.** Event IDs, `nltest` output, and the `DisabledComponents` bit meanings reflect

current Microsoft documentation — **verify them for your OS build** rather than assuming.

The standard diagnostics model

This solution uses **one** repeatable model across our catalog:

> ***On a domain-joined Windows endpoint, collect every secure-channel / Netlogon signal read-only, diagnose the specific root cause of Event ID 5719, repair it safely (preview-first), keep IPv6 in a supported dual-stack configuration, then validate and prevent recurrence — locally or via Intune Proactive Remediations.***

Every step follows the repeatable pattern: **Assess** → **Diagnose** → **Repair** → **Validate** → **Prevent.**

2. Requirements — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, pilot group, network, Windows baseline) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to running this **endpoint diagnostics toolkit** for domain secure-channel / Netlogon health.

> ***Scope reminder.*** *These requirements are for running **endpoint PowerShell** on **domain-joined Windows devices** — locally or via **Intune Proactive Remediations**. They deliberately do **not** include building or diagnosing the domain controllers themselves.*

R1. Domain-joined Windows devices

Description: The target devices are joined to an on-premises Active Directory domain (hybrid-joined is fine; the secure channel is the AD machine-account trust). **Why:** Event ID 5719 and the secure channel only exist on **AD domain-joined** members; the scripts short-circuit and report "not domain-joined" otherwise. **Verify:** On the device run `(Get-CimInstance Win32_ComputerSystem).PartOfDomain` — it returns **True**; or `systeminfo` shows the **Domain**. **Implement:** Join the device to the domain (**Settings** → **Accounts** → **Access work or school**, or `Add-Computer`) if it should be a member; Entra-only (AAD-joined, no on-prem AD) devices are out of scope for the secure-channel checks.

R2. Local administrator / SYSTEM execution context

Description: The scripts run **elevated** locally, or as **SYSTEM** when deployed via Proactive Remediations. **Why:** Reading the System event log, querying the secure channel, resetting time, and writing the IPv6 registry value require elevation. **Verify:** Launch PowerShell **as administrator** (locally); for Proactive Remediations the platform runs the scripts as SYSTEM by default. **Implement:** Right-click **Run as administrator** for local runs; for at-scale runs use the Proactive Remediations packaging in the Implementation Guide (which runs as SYSTEM).

R3. Windows PowerShell 5.1+ with in-box modules

Description: Windows PowerShell **5.1** (or PowerShell 7) with the in-box `DnsClient` cmdlets (`Resolve-DnsName`), CIM, and `Microsoft.PowerShell.Archive` (`Compress-Archive`). **Why:** The toolkit uses **only in-box** Windows capabilities — no modules to install on endpoints — so it runs unchanged under Proactive Remediations. **Verify:** `$PSVersionTable.PSVersion` shows 5.1+; `Get-Command Resolve-DnsName, Compress-Archive` resolve. **Implement:** These ship with Windows 10/11 and Windows Server 2016+. Ensure the device meets the supported Windows baseline

(Global Requirement **G6**).

R4. In-box domain tooling present (`nltest`, `w32tm`)

Description: The built-in `nltest.exe` and `w32tm.exe` utilities are available on the device. **Why:** Discovery/Validation use `nltest /sc_query` and `/dsgetdc` to view the DC-locator/secure channel and `w32tm` to read and resync time. **Verify:** `where.exe nltest` and `where.exe w32tm` both return a path under `System32`. **Implement:** Both ship in Windows by default; if `nltest` is missing on a client SKU, install the **RSAT: Active Directory** feature, or rely on the secure-channel and DNS checks that do not need it.

R5. Network path to a domain controller (for repair)

Description: The device can reach a DC on **88 (Kerberos)**, **135 (RPC EPM)**, **389 (LDAP)**, and **445 (SMB)**. **Why:** Diagnosis works offline, but **repair** (`Test-ComputerSecureChannel -Repair`, `nltest /sc_reset`, `w32tm /resync`) requires a reachable DC. Blocked ports are themselves a common 5719 cause. **Verify:** Run `NETLOGONDX-Discovery.ps1` and read the **DC ports reachable (88/135/389/445)** line, or `Test-NetConnection <dc> -Port 389`. **Implement:** On-network, unblock those ports to the DCs / correct the VLAN/firewall; off-network, connect the corporate VPN/DirectAccess before repairing. Do **not** block AD ports between members and DCs.

R6. Correct internal DNS on the device NIC

Description: The device's DNS servers are the **internal AD DNS** servers that host the domain's SRV locator records. **Why:** The DC Locator finds DCs via `_ldap._tcp.dc._msdcs.<domain>`; a NIC pointed at a public resolver cannot resolve it, causing 5719. **Verify:** `Get-DnsClientServerAddress`; and `Resolve-DnsName _ldap._tcp.dc._msdcs.<domain> -Type SRV` returns DC targets (Discovery reports this). **Implement:** Set the NIC (or DHCP scope) to the internal AD DNS servers; remove public resolvers from domain members. Re-run `ipconfig /registerdns` after correcting.

R7. Intune Proactive Remediations available (for at-scale runs)

Description: An Intune plan that includes **Proactive Remediations** (Endpoint analytics) — Windows Enterprise E3/E5, Windows 365 Enterprise, or equivalent — for fleet-wide detection/remediation. **Why:** At scale, `NETLOGONDX-Validation.ps1` is the **detection** script and `NETLOGONDX-Remediation.ps1` the **remediation** script; this needs the Proactive Remediations feature and enrolled devices (Global Requirement **G4**). **Verify:** Intune admin center → **Reports** → **Endpoint analytics** → **Proactive remediations** is available. **Implement:** Assign a qualifying license and onboard Endpoint analytics. Local runs do **not** need this — you can triage a single device with the scripts alone.

R8. A place to collect output and escalation bundles

Description: The on-device folder `C:\ProgramData\NETLOGONDX` (created automatically) and a way to retrieve files from endpoints for fleet roll-up / escalation. **Why:** Report writes a per-device CSV and Collect writes an escalation ZIP there; you gather them to measure coverage and to attach to support cases. **Verify:** After a run, the folder contains `NETLOGONDX-Report.csv` and/or a `NETLOGONDX-Collect-*.zip`. **Implement:** Use your existing endpoint file-collection method (Intune device query/collection, config-manager, or manual) to retrieve the artifacts. Nothing sensitive is written

into the bundle.

R9. Pilot device and pilot group

Description: At least one affected/representative domain-joined pilot device, and (for at-scale) the `NETLOGONDX-Pilot` Entra security group. **Why:** Every solution is validated on a pilot before fleet rollout, and the Remediation script is proven in `-WhatIf` preview first. **Verify:** The pilot device is enrolled (G4) and domain-joined (R1); the pilot group exists (Global Requirement **G5**). **Implement:** Create the `NETLOGONDX-Pilot` security group (Entra → **Groups** → **New group** → **Security**) and add the pilot device(s); assign the Proactive Remediations package to it first.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups**

→ **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Follow these steps **in order**. The *why* for each check lives in the Solution Overview; the steps below are deliberately short and exact so you can **click (or run) to implement**. Throughout, the repeatable pattern applies: **Assess** → **Diagnose** → **Repair** → **Validate** → **Prevent.** Prove everything on a **pilot device** (and the `NETLOGONDX-Pilot` group) before you widen.

All scripts write to `C:\ProgramData\NETLOGONDX` and use **only in-box Windows commands** — nothing to install on endpoints. Run them **elevated** locally, or as **SYSTEM** via Intune Proactive Remediations. The Intune admin center is `https://intune.microsoft.com`.

> ***This is a diagnostics toolkit**, not a configuration baseline. Only* `NETLOGONDX-Remediation.ps1` *changes anything, and it **defaults to** `-WhatIf` (preview). Resolve the two findings — **Event ID 5719 (Netlogon)** and **IPv6 recommended configuration** — the same way every time.*

Phase 1 — Assess (get the scripts onto the device)

Step 1. Stage the toolkit

1. Copy the `NETLOGONDX-Scripts.zip` contents to the device (or a share), e.g. to `C:\Temp\NETLOGONDX`.
2. Open **Windows PowerShell as administrator** (Start → type *PowerShell* → **Run as administrator**).
3. `cd C:\Temp\NETLOGONDX`.

Step 2. Confirm the device is in scope

1. Run `(Get-CimInstance Win32_ComputerSystem).PartOfDomain` — it must return **True** (R1).

2. Confirm you are elevated (R2) and `where.exe nltest w32tm` both resolve (R4).

Phase 2 — Diagnose (collect every signal, read-only)

Step 3. Run Discovery

1. Run:

```
powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Discovery.ps1
```

1. Read the PASS/FAIL/INFO table. It reports, in one pass: recent **Event ID 5719** (and related 5722 / 5723 / 3210 events), **secure channel** (`Test-ComputerSecureChannel`), **DC locator** (`nltest /sc_query`, `/dsgetdc`), **DNS SRV** resolution of `_ldap._tcp.dc._msdcs.<domain>`, **time sync** (`w32tm`), **DC ports 88/135/389/445**, and **IPv6** `DisabledComponents`.

1. The full detail is saved to `C:\ProgramData\NETLOGONDX\NETLOGONDX-Discovery.json` and `…\NETLOGONDX-Discovery.txt`.

Step 4. Read the root cause off the table

Map the failing signal to the cause (Solution Overview has the full tree):

- **DNS SRV = FAIL** → wrong/unreachable DNS or missing SRV records (R6).
- **Secure channel = FAIL** → machine-account/trust password out of sync (snapshot/clone/offline).
- **Time = FAIL** → Kerberos clock skew > 5 minutes.
- **DC ports = FAIL** → firewall/VLAN blocking 88/135/389/445, or no DC reachable (R5).
- **IPv6 = FAIL** → invalid `DisabledComponents` (e.g. `0xffffffff`).
- **5719 present but everything else PASS** → likely the **boot-time NIC/power race** (see Phase 5).

Phase 3 — Repair (preview first, then apply)

Step 5. Preview the remediation

1. Run the remediation in its **default preview** mode (changes nothing):

```
powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1
```

1. Read exactly what it **would** do: repair the secure channel (`Test-ComputerSecureChannel -Repair`, falling back to `nltest /sc_reset:<domain>`), start Windows Time and `w32tm /resync /force`, and — **only if the IPv6 value is invalid** — reset `DisabledComponents` to `0x0`. It **never disables IPv6**.

Step 6. Apply the remediation

1. When the preview looks right, apply:

```
powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1
-WhatIf:$false
```

1. If the secure-channel repair needs domain credentials, pass them:

```
.\NETLOGONDX-Remediation.ps1 -WhatIf:$false -Credential (Get-Credential)
```

1. If **IPv6 was corrected**, note that a **restart** is required for the change to take effect.

> ***DNS / ports first when they are the cause.*** *If Discovery showed **DNS SRV** or **DC ports** failing, fix those at the network/NIC layer (R5/R6) *before* re-running repair — a secure-channel reset cannot succeed with no DC path. Correct the NIC DNS, then* `ipconfig /flushdns; ipconfig /registerdns`*, and re-run Discovery.*

Phase 4 — Validate

Step 7. Confirm healthy

1. Run:

```
powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Validation.ps1
```

1. Confirm **OVERALL: HEALTHY** (exit code **0**): secure channel healthy, time in sync, DNS resolves

DCs, IPv6 not invalidly disabled.

1. Produce the summary for the record:

`powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Report.ps1` — reads `C:\ProgramData\NETLOGONDX\NETLOGONDX-Report.csv` / `.txt`.

Step 8. Escalate if still failing

1. If a signal still fails, bundle everything for support:

```
powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Collect.ps1
```

1. Attach the resulting `NETLOGONDX-Collect-<host>-<timestamp>.zip` to your ticket (it contains no

credentials or secrets).

Phase 5 — Deploy at scale via Intune Proactive Remediations (and Prevent)

Step 9. Create the script package

1. In the Intune admin center, go to **Reports → Endpoint analytics → Proactive remediations →

Create script package**.

1. **Basics** → **Name:** `NETLOGONDX – Secure-Channel & Netlogon Health`; add a description, e.g.

Detects Netlogon 5719 / broken secure channel, clock skew, DC-locator DNS failures, and invalid IPv6 config; repairs safely. Click **Next**.

1. **Settings:**

• **Detection script file:** upload `NETLOGONDX-Validation.ps1` (exits `1` when unhealthy).

• **Remediation script file:** upload `NETLOGONDX-Remediation.ps1`. In the packaged remediation

copy, ensure it will **apply** under SYSTEM (either leave the default so it applies, or add `-Force`/`$WhatIfPreference=$false` at the top of the packaged copy — the shipped script previews by default for safe manual runs).

- **Run this script using the logged-on credentials: No** (runs as SYSTEM).
- **Enforce script signature check: No** (unless you sign the scripts).
- **Run script in 64-bit PowerShell: Yes**.

Click **Next**.

1. **Scope tags** (set if you use them) → **Next**.

Step 10. Assign to the pilot group first

1. On **Assignments**, add the `NETLOGONDX-Pilot` group (Global Requirement **G5** / R9).
2. Set a **daily** schedule (Endpoint analytics → the package → **Properties → Assignments →

schedule**) so detection re-runs and confirms devices stay healthy.

1. **Review + create** → **Create.**

Step 11. Read results and expand

1. Open **Reports → Endpoint analytics → Proactive remediations → `NETLOGONDX – Secure-Channel &

Netlogon Health` **and review** Device status**: *Without issues* vs *Issues detected* / *Issues remediated*.

1. Once the pilot passes (Testing gate), broaden the assignment in waves to the next group.
2. **Prevent recurrence** (the boot-time NIC/power race and drift) — apply the standing controls in

Best Practices: disable NIC **"Allow the computer to turn off this device to save power,"** set **"Always wait for the network at computer startup and logon,"** keep domain members on **internal AD DNS** and a valid **time hierarchy**, and keep **IPv6 enabled** (`0x0`). Re-run the package each wave and confirm **Without issues** matches the assigned devices before widening.

5. Testing & Validation — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Validate against a **pilot device** (and the `NETLOGONDX-Pilot` group) before expanding. Each test has an explicit **pass/fail gate**. Use the included endpoint PowerShell scripts for the checks; the device-level steps confirm the real user-facing behavior.

> ***What "pass" means here:*** *the toolkit correctly *identifies* the root cause of Event ID 5719, *repairs* it safely, leaves **IPv6 in a supported configuration**, and reports **HEALTHY**. This is a diagnostics toolkit — it does not enroll devices or change how they are joined.*

Test 1 — Discovery collects every signal

Procedure: Run `NETLOGONDX-Discovery.ps1` elevated on the pilot device. **Pass:** The table shows a line for **each** signal — 5719 events, secure channel, DC locator, DNS SRV, time sync, DC ports (88/135/389/445), IPv6 `DisabledComponents` — and `C:\ProgramData\NETLOGONDX\NETLOGONDX-Discovery.json` is written. **Fail:** Any signal missing/errors out → confirm elevation (R2) and that `nltest`/`w32tm` resolve (R4).

Test 2 — Validation gate and exit code

Procedure: Run `NETLOGONDX-Validation.ps1`; then check the exit code (`$LASTEXITCODE`). **Pass:** On a healthy device, **OVERALL: HEALTHY** and exit code **0**. On a device with a known issue, the failing check(s) are listed and exit code is **1**. **Fail:** Exit code does not reflect health (e.g. 0 while a check reads FAIL) → re-run elevated and capture output for Collect.

Test 3 — Secure-channel repair (preview then apply)

Procedure: On a device with a broken secure channel (`Test-ComputerSecureChannel` = False), run `NETLOGONDX-Remediation.ps1` (preview), then `-WhatIf:$false`. **Pass:** Preview reports it **would** repair and changes nothing; apply runs `Test-ComputerSecureChannel -Repair` (or falls back to `nltest /sc_reset`) and a follow-up `Test-ComputerSecureChannel` returns **True**. **Fail:** Repair errors → confirm a DC is reachable on 88/135/389/445 (R5) and supply `-Credential`.

Test 4 — Time re-sync

Procedure: On a device with clock skew, run the remediation (`-WhatIf:$false`), then re-run Validation. **Pass:** `w32tm /resync` succeeds, the measured offset drops **below 5 minutes**, and the **Time in sync** check reads PASS. **Fail:** Offset still high → confirm the Windows Time service is running and the domain time hierarchy is healthy; check the time source in `w32tm /query /status`.

Test 5 — DNS DC-locator resolution

Procedure: Run Discovery and read the **DNS resolves domain SRV records** line; compare the client's DNS servers to the internal AD DNS. **Pass:** `_ldap._tcp.dc._msdcs.<domain>` resolves to DC targets and the NIC uses internal AD DNS. **Fail:** Not resolved / public resolver in use → correct NIC DNS (R6), then `ipconfig /flushdns; ipconfig /registerdns` and re-test.

Test 6 — IPv6 recommended configuration

Procedure: On a device with an **invalid** value (set `DisabledComponents = 0xffffffff` on a lab device), run Discovery, then the remediation (preview, then `-WhatIf:$false`), then reboot and re-run Validation. **Pass:** Discovery flags the value **INVALID/unsupported**; remediation resets it to `0x0` (never disabling IPv6); after reboot the **IPv6 not invalidly disabled** check reads PASS. **Expected variance:** a **supported** value (`0x20` or `0x11`) is reported as acceptable and is **left unchanged** — that is correct, not a failure. **Fail:** Remediation disables IPv6 or changes a supported value → it must not; re-confirm you are on the shipped script.

Test 7 — DC port reachability

Procedure: Run Discovery and read the **DC ports reachable (88/135/389/445)** line. **Pass:** All four ports show **open** to the located DC (or the specific blocked port is correctly reported). **Fail:** A port shows **blocked** unexpectedly → verify firewall/VLAN rules to the DCs (R5), then re-test.

Test 8 — Report and escalation bundle

Procedure: Run `NETLOGONDX-Report.ps1`, then `NETLOGONDX-Collect.ps1`. **Pass:** `NETLOGONDX-Report.csv` (one row) and `NETLOGONDX-Report.txt` are written with an OVERALL verdict; Collect produces `NETLOGONDX-Collect-<host>-<timestamp>.zip` containing the events, `nltest`, `ipconfig`, `w32tm`, and IPv6 artifacts — with **no credentials/secrets**. **Fail:** Missing artifacts → confirm write access to `C:\ProgramData\NETLOGONDX` (R8).

Test 9 — Proactive Remediations at scale

Procedure: Deploy the `NETLOGONDX – Secure-Channel & Netlogon Health` package (detection = Validation, remediation = Remediation) to the `NETLOGONDX-Pilot` group; wait for a detection cycle. **Pass:** Endpoint analytics → the package → **Device status** shows healthy devices as *Without issues* and previously-broken devices moving to *Issues remediated*. **Fail:** Devices stuck in *Issues detected* → open a device, confirm SYSTEM context and 64-bit PowerShell in the package settings, and that the packaged remediation applies (not left in preview).

Test 10 — No enrollment/join change (negative test)

Procedure: Confirm the device is still domain-joined and enrolled exactly as before, after running the toolkit. **Pass:** Domain membership and Intune enrollment are **unchanged**; the toolkit only diagnosed/repaired the secure channel, time, DNS use, and IPv6 value. **Fail:** Any join/enrollment change → the toolkit does not do this; investigate other tooling.

Acceptance gate

Proceed to **Expand** only when Tests 1–8 pass on the pilot device, Test 9 confirms the Proactive Remediations package behaves at scale, and Test 10 confirms enrollment/join is unchanged.

6. Troubleshooting — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Work top-down: run **Discovery** first, then read the failing signal and fix its cause. Most 5719 cases are one of **DNS, secure channel, time, blocked ports, no DC path, the boot-time NIC race, or an invalid IPv6 value** — Discovery tells you which. Each symptom below gives the **likely cause** and the **fix**. (Prevention lives in Best Practices.)

> ***Before you start:*** *confirm this is a *client-side* secure-channel problem. If **every** domain member on a subnet fails at once, suspect the **DCs / DNS / network** (a DC or AD-DNS outage), not each endpoint — this toolkit diagnoses endpoints, not the domain controllers.*

Symptom: Event ID 5719 keeps appearing, but Validation says HEALTHY

Likely cause: A **boot-time race** — the NIC (or Wi-Fi/VPN) isn't up when Netlogon starts, so the first secure-session attempt fails and logs 5719, then succeeds moments later. **Fix:**

1. Confirm the 5719 events cluster at **startup** (Discovery lists event times).
2. Disable NIC power-off: **Device Manager** → **(NIC)** → **Properties** → **Power Management** → uncheck

"Allow the computer to turn off this device to save power."

1. Set the GPO **Computer Configuration → Administrative Templates → System → Logon → "Always wait for

the network at computer startup and logon" = Enabled.**

1. These transient boot 5719s are cosmetic once logon succeeds; the controls above stop them.

Symptom: `Test-ComputerSecureChannel` returns False (trust broken)

Likely cause: The machine-account password is out of sync — device restored from a **snapshot/ image**, **cloned** without sysprep, or **off the domain longer than the machine-password age**. **Fix:**

1. Ensure a DC is reachable (Discovery **DC ports** all open — R5).
2. Run `NETLOGONDX-Remediation.ps1` (preview), then `-WhatIf:$false` — it runs

`Test-ComputerSecureChannel -Repair`, falling back to `nltest /sc_reset:<domain>`.

1. If repair needs credentials, add `-Credential (Get-Credential)`.
2. Re-run `NETLOGONDX-Validation.ps1`; the secure-channel check should read PASS. Avoid the old

"unjoin/rejoin" ritual — repair/reset is faster and keeps the computer object.

Symptom: DNS SRV records do not resolve (DC Locator fails)

Likely cause: The NIC points at a **public/wrong DNS server**, or the domain SRV records are missing. **Fix:**

1. Discovery shows the client DNS servers and the `_ldap._tcp.dc._msdcs.<domain>` result.
2. Set the NIC (or DHCP) to the **internal AD DNS** servers only (R6); remove public resolvers from

domain members.

1. `ipconfig /flushdns; ipconfig /registerdns`, then re-run Discovery.
2. If internal DNS still lacks the SRV records, that is a **DC/DNS-side** issue (escalate with the

Collect bundle).

Symptom: Clock skew — time is more than 5 minutes off

Likely cause: Drifted clock (dead CMOS battery, wrong/blocked time source) breaks Kerberos. **Fix:**

1. Run the remediation (`-WhatIf:$false`) — it starts Windows Time and runs `w32tm /resync /force`.
2. Confirm the source: `w32tm /query /status` should show the domain hierarchy (not `Local CMOS

Clock`).

1. Ensure UDP/123 to the time source is allowed; replace the CMOS battery if the clock keeps drifting.

Symptom: A DC port shows "blocked" (88/135/389/445)

Likely cause: Firewall/VLAN/segmentation change, host firewall, or the device is off-network. **Fix:**

1. From the device, confirm with `Test-NetConnection <dc> -Port 389` (repeat for 88/135/445).
2. Unblock those ports to the DCs, or connect the corporate VPN/DirectAccess if off-network (R5).
3. Re-run Discovery; repair only after the ports are open — a secure-channel reset cannot reach a blocked DC.

Symptom: IPv6 value flagged INVALID / unsupported

Likely cause: `DisabledComponents` is set to an unsupported value such as `0xffffffff` (or the blanket-disable `0xff`) — often a copy-pasted "disable IPv6" tweak. **Fix:**

1. Run the remediation (preview, then `-WhatIf:$false`) — it resets the value to `0x0` (IPv6 enabled, dual-stack) and **never disables IPv6**.
1. **Restart** the device for the change to take effect, then re-run Validation.
2. Do not re-apply a "disable IPv6" fix: it breaks **DirectAccess / Remote Assistance / HomeGroup** and is **unsupported by Microsoft testing**. If you have a genuine need to prefer IPv4, use the supported `0x20` (prefer IPv4, IPv6 still enabled) — the toolkit leaves that value untouched.

Symptom: Remediation says "Preview only" and changed nothing

Likely cause: Expected — the script **defaults to** `-WhatIf`. **Fix:** Re-run with `-WhatIf:$false` (or `-Force`) to apply. In a Proactive Remediations package, ensure the packaged remediation copy applies under SYSTEM (not left in preview).

Symptom: Works on one device, not another

Likely cause: The failing device differs on one signal (DNS, ports, time, or IPv6). **Fix:**

1. Run Discovery on **both** and compare the tables — the difference is the cause.
2. Fix the specific failing signal per the sections above, then re-validate.

Symptom: Proactive Remediations shows "Issues detected" but never remediates

Likely cause: Detection exits `1` (correctly) but the packaged remediation is still previewing, or context/architecture is wrong. **Fix:**

1. Confirm the package uses **Run as SYSTEM = Yes** and **64-bit PowerShell = Yes**.

2. Confirm the packaged **remediation** script applies (not `-WhatIf`).
3. Check the device is reachable to a DC (repair needs a DC path — R5).

Escalation checklist

Before opening a Microsoft support case, run `NETLOGONDX-Collect.ps1` and attach the ZIP. It contains: the relevant System-log events (NETLOGON / Time-Service / Kerberos), `nltest /sc_query`, `/dsgetdc`, `/dclist`, `ipconfig /all`, the client DNS list, the SRV-record result, `w32tm` status and configuration, the secure-channel state, and the IPv6 `DisabledComponents` value — with **no credentials or secrets**.

7. Best Practices — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

These practices keep the toolkit safe, repeatable, and effective — and, just as important, **prevent** Event ID 5719 from recurring. They assume the diagnostics model in the Solution Overview: read-only collection first, preview-first repair, IPv6 kept in a supported dual-stack configuration, run locally or via Intune Proactive Remediations.

Diagnose before you touch anything

- **Always run Discovery first.** Read the failing signal off the table; don't reset the secure channel

on a hunch when the real cause is DNS or a blocked port.

- **Fix the network layer before repairing the trust.** If **DNS SRV** or **DC ports** are failing,

correct those first — `Test-ComputerSecureChannel -Repair` and `nltest /sc_reset` cannot succeed with no DC path.

Repair safely (preview-first)

- **Preview every change.** `NETLOGONDX-Remediation.ps1` **defaults to** `-WhatIf`. Read the plan, then

re-run with `-WhatIf:$false` (or `-Force`) to apply.

- **Prefer repair over rejoin.** `Test-ComputerSecureChannel -Repair` / `nltest /sc_reset` fix the

trust **without** deleting the computer object or re-enrolling — faster and less disruptive than unjoin/rejoin.

- **Supply credentials only when needed.** Pass `-Credential` for the secure-channel repair only if the

device's current context can't authenticate; never hard-code credentials into a packaged script.

Keep IPv6 in a supported configuration (never blanket-disable)

- **Leave IPv6 ENABLED.** The recommended `DisabledComponents` value is `0x0` (or absent) — full

dual-stack. This is what Microsoft tests and supports.

- **Only correct invalid values.** Reset unsupported values (e.g. `0xffffffff`, `0xff`) back to `0x0`. The toolkit does exactly this and **never disables IPv6**.
- **Use supported non-default values deliberately.** If you must prefer IPv4, use `0x20` (prefer IPv4, IPv6 still enabled); `0x11` (disable IPv6 on non-tunnel + tunnel interfaces) only with a documented justification. Record *why* you chose it.
- **Know the blast radius.** Disabling IPv6 breaks **DirectAccess, Remote Assistance, and HomeGroup** and is **unsupported by Microsoft's testing** — never present "just disable IPv6" as a fix.
- **Remember the restart.** An IPv6 `DisabledComponents` change takes effect only after a **reboot**.

Prevent recurrence (the boot-time race and drift)

- **Stop the NIC power-off.** Uncheck **"Allow the computer to turn off this device to save power"** on the network adapter (deploy via policy for the fleet) so the link is up when Netlogon starts.
- **Wait for the network at boot.** Enable the GPO **"Always wait for the network at computer startup and logon"** so early logon has a DC path — this eliminates most transient boot 5719s.
- **Pin domain members to internal AD DNS.** No public resolvers on domain members; the DC Locator depends on the internal SRV records.
- **Keep a healthy time hierarchy.** Domain members should sync from the domain (ultimately the PDC emulator), not `Local CMOS Clock`; replace failing CMOS batteries.

Operate at scale with Proactive Remediations

- **Detection = Validation, Remediation = Remediation.** Package `NETLOGONDX – Secure-Channel & Netlogon Health` with Validation (exits `1` when unhealthy) as detection and Remediation as the fix; run as **SYSTEM**, **64-bit**.
- **Pilot first, expand in waves.** Assign to `NETLOGONDX-Pilot` first; only broaden after the Testing gate passes. Schedule detection **daily** so drift is caught early.
- **Measure with the CSV.** Harvest `NETLOGONDX-Report.csv` from devices to see coverage before and after a wave; only widen when *Without issues* matches the assigned devices.

Keep evidence and change control

- **Bundle for escalation.** Run `NETLOGONDX-Collect.ps1` and attach the ZIP to any support case — it captures the events, `nltest`, `ipconfig`, `w32tm`, and IPv6 state, with **no secrets**.
- **Record what you changed and why.** Note the failing signal, the fix applied, and (for IPv6) the

before/after value and justification for any non-default supported value.

- **Re-validate after OS updates.** Confirm the checks still pass and the IPv6 value holds after major Windows feature updates before widening further.

8. Deliverables Summary — Windows Domain Secure-Channel & Netlogon Health Diagnostics Toolkit

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

- **Executive Summary** — the business case and outcome in one page, and the explicit scope: an endpoint diagnostics toolkit for Netlogon 5719 / broken secure channel and IPv6 recommended configuration — not a config baseline and not a domain-controller health tool.
- **Solution Overview** — what Event ID 5719 means, the full **root-cause tree** (DNS/DC-locator, secure channel/trust, time skew, blocked DC ports, no DC path, the boot-time NIC/power race, and invalid IPv6), the **IPv6 recommended-configuration** guidance (keep enabled; correct invalid `DisabledComponents` only), and how to run locally vs via Intune Proactive Remediations.
- **Requirements Guide** — every device/tooling/network prerequisite with how to **verify** and how to **implement if missing**, plus the category-level Global Requirements.
- **Implementation Guide** — exact, ordered steps across five phases: Assess (stage the scripts), Diagnose (Discovery), Repair (preview-first Remediation), Validate (Validation/Report/Collect), and deploy at scale via **Proactive Remediations** with full navigation paths — and Prevent.
- **Testing & Validation** — ten tests with explicit pass/fail gates (including the IPv6 correction and a negative test that join/enrollment is unchanged) and an acceptance gate.
- **Troubleshooting** — Discovery-first diagnosis for each 5719 cause: boot race, broken trust, DNS, time, blocked ports, invalid IPv6, and Proactive Remediations behavior.
- **Best Practices** — diagnose-before-touch, preview-first repair, the correct IPv6 posture, recurrence prevention (NIC power-off, wait-for-network, internal DNS, time hierarchy), and operating at scale.

PowerShell scripts (endpoint) — `NETLOGONDX-Scripts.zip`

All scripts run **locally on the affected Windows device** (elevated) or as **SYSTEM via Intune Proactive Remediations**, use **only in-box Windows commands** (no modules to install), and write to `C:\ProgramData\NETLOGONDX`. Each has a header explaining **Purpose · How to run · Expected results**. Discovery, Validation, Report, and Collect are **read-only**; Remediation **defaults to** `-WhatIf` (preview first, then apply).

1. `NETLOGONDX-Discovery.ps1` — one-pass collector: recent Event ID 5719 + related secure-channel

events, `Test-ComputerSecureChannel`, `nltest /sc_query` and `/dsgetdc`, DNS SRV resolution of the DC-locator records, `w32tm` time state, DC port reachability (88/135/389/445), and the IPv6 `DisabledComponents` value. Writes a JSON + transcript. **Read-only.**

1. `NETLOGONDX-Validation.ps1` — pass/fail gate asserting secure channel healthy, time in sync, DNS

resolves DCs, and IPv6 not invalidly disabled. Doubles as the Proactive Remediations **detection** script (exit `0` healthy / exit `1` issues). **Read-only.**

1. `NETLOGONDX-Report.ps1` — readable + one-row-CSV health summary across all signals for fleet

roll-up. **Read-only.**

1. `NETLOGONDX-Remediation.ps1` — safe repair: `Test-ComputerSecureChannel -Repair` (falling back

to `nltest /sc_reset`), start Windows Time + `w32tm /resync`, and **correct an invalid IPv6** `DisabledComponents` **back to** `0x0` (never disables IPv6). `-WhatIf` **by default**; applies only with `-WhatIf:$false` (or `-Force`). The Proactive Remediations **remediation** script.

1. `NETLOGONDX-Collect.ps1` — bundles the System-log events, `nltest`, `ipconfig`, `w32tm`, and

IPv6 artifacts into one timestamped ZIP for escalation. **Read-only; contains no secrets.**

Hero image

- `NETLOGONDX-hero.png` — the product image used on the marketplace listing.

How it's delivered

A **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NETLOGONDX-Collect.ps1

Purpose: Gathers everything a support engineer (or a Microsoft support case) needs to diagnose a stubborn Netlogon 5719 / secure-channel problem, and packages it into a single timestamped ZIP you can attach to a ticket. It collects (read-only): - the relevant System-log events (NETLOGON, Time-Service, and Kerberos client events), - nltest output (/sc_query, /dsgetdc, /dclist), - ipconfig /all and the client DNS server list, - w32tm /query status and configuration, - the IPv6 DisabledComponents registry value, - the JSON/CSV/TXT output from the other NETLOGONDX scripts if present. It changes nothing on the device — it only reads and copies. No credentials or secrets are written into the bundle. HOW TO RUN Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Collect.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 30 At scale: run under SYSTEM via Intune Proactive Remediations, then retrieve the ZIP from the device (e.g. with a follow-up collection package or your endpoint tooling). EXPECTED RESULTS A ZIP at <OutputDir>\NETLOGONDX-Collect-<host>-<timestamp>.zip containing the text/CSV/JSON artifacts above, and the path printed to the console. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 30)

How to run: Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Collect.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 30 At scale: run under SYSTEM via Intune Proactive Remediations, then retrieve the ZIP from the device (e.g. with a follow-up collection package or your endpoint tooling). EXPECTED RESULTS A ZIP at <OutputDir>\NETLOGONDX-Collect-<host>-<timestamp>.zip containing the text/CSV/JSON artifacts above, and the path printed to the console. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 30)

NETLOGONDX-Discovery.ps1

Purpose: Runs LOCALLY on an affected domain-joined Windows device and gathers the full root-cause picture behind Netlogon Event ID 5719 ("There are currently no logon servers available" / broken secure channel) in one pass: - recent NETLOGON Event ID 5719 and related secure-channel events from the System log, - the machine secure channel state (Test-ComputerSecureChannel), - the domain-controller locator view (nltest /sc_query and nltest /dsgetdc), - DNS resolution of the domain SRV locator records (_ldap._tcp.dc._msdcs.<domain>), - the Windows Time (w32tm) synchronization state and source, - TCP reachability to a domain controller on ports 88/135/389/445, - the IPv6 DisabledComponents registry value under Tcpip6\Parameters. This is a DIAGNOSTICS collector — it changes nothing. Pair it with NETLOGONDX-Validation.ps1 (pass/fail gate) and NETLOGONDX-Remediation.ps1 (safe, -WhatIf-by-default repair). HOW TO RUN Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Discovery.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 14 At scale: run under SYSTEM via Intune Proactive Remediations, or as a one-off with any endpoint management tool. No modules required — everything uses in-box Windows commands. EXPECTED RESULTS A per-signal PASS/FAIL/INFO table and an overall summary printed to the console, plus a machine-readable copy written to <OutputDir>\NETLOGONDX-Discovery.json and a

readable transcript at <OutputDir>\NETLOGONDX-Discovery.txt. A healthy device shows: no recent 5719, secure channel OK, DCs resolvable, time in sync, DC ports open, IPv6 not invalidly disabled. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 14)

How to run: Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Discovery.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 14 At scale: run under SYSTEM via Intune Proactive Remediations, or as a one-off with any endpoint management tool. No modules required — everything uses in-box Windows commands. EXPECTED RESULTS A per-signal PASS/FAIL/INFO table and an overall summary printed to the console, plus a machine-readable copy written to <OutputDir>\NETLOGONDX-Discovery.json and a readable transcript at <OutputDir>\NETLOGONDX-Discovery.txt. A healthy device shows: no recent 5719, secure channel OK, DCs resolvable, time in sync, DC ports open, IPv6 not invalidly disabled. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 14)

NETLOGONDX-Remediation.ps1

Purpose: Repairs the common root causes behind Netlogon Event ID 5719 / a broken secure channel, in a safe, reviewable order. It: 1. repairs the machine secure channel (Test-ComputerSecureChannel -Repair; if that needs credentials/fails, resets it with nltest /sc_reset), 2. re-synchronizes Windows Time with the domain hierarchy (w32tm /resync, service started), 3. CORRECTS an INVALID IPv6 DisabledComponents value back to the recommended 0x0 (dual-stack default) — it never disables IPv6. IPv6 policy: IPv6 must stay ENABLED (dual-stack). This script only fixes UNSUPPORTED values (e.g. 0xffffffff, 0xff); it leaves the supported values 0x0 / 0x20 / 0x11 untouched. Disabling IPv6 breaks DirectAccess, Remote Assistance, and HomeGroup and is not covered by Microsoft's testing — so this tool will not do it. DEFAULTS TO -WhatIf: with no switch it PREVIEWS every change and modifies nothing. It applies only when you pass -WhatIf:$false (or -Force). Suitable as an Intune Proactive Remediations REMEDIATION script paired with NETLOGONDX-Validation.ps1 as the detection script. HOW TO RUN Preview (default — no changes): powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1 Apply the repairs: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1 -WhatIf:$false As a Proactive Remediations remediation script (runs as SYSTEM) it applies automatically — set -Apply behavior by leaving -WhatIf:$false in the packaged script body if desired. EXPECTED RESULTS In preview: a report of what is unhealthy and exactly what WOULD be repaired. When applied: secure channel repaired/reset, time resynced, and any invalid IPv6 value reset to 0x0. Re-run NETLOGONDX-Validation.ps1 to confirm HEALTHY. A restart may be required for an IPv6 change to take effect. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([pscredential]$Credential, [switch]$Force)

How to run: Preview (default — no changes): powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1 Apply the repairs: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Remediation.ps1 -WhatIf:$false As a Proactive Remediations remediation script (runs as SYSTEM) it applies automatically — set -Apply behavior by leaving -WhatIf:$false in the packaged script body if desired. EXPECTED RESULTS In preview: a report of what is unhealthy and exactly what WOULD be repaired. When applied: secure channel repaired/reset, time resynced, and any invalid IPv6 value reset to 0x0. Re-run NETLOGONDX-Validation.ps1 to confirm HEALTHY. A restart may be required for an IPv6 change to take effect. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param(

[pscredential]$Credential, [switch]$Force)

NETLOGONDX-Report.ps1

Purpose: Produces a single, human-readable health summary of a device's domain secure-channel and Netlogon posture, and writes a one-row CSV suitable for fleet roll-up (collect the CSVs from many devices to see coverage before/after a remediation wave). It reports the same signals the Discovery collector gathers — secure channel, DC locator, DNS SRV, time sync, DC port reachability (88/135/389/445), IPv6 DisabledComponents, and recent Event ID 5719 count — as a compact Healthy/Attention verdict per signal. Read-only. HOW TO RUN Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Report.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 14 At scale: run under SYSTEM via Intune Proactive Remediations (as an additional detection package) and harvest <OutputDir>\NETLOGONDX-Report.csv from each device. EXPECTED RESULTS A formatted summary printed to the console and written to <OutputDir>\NETLOGONDX-Report.txt, plus a single-row <OutputDir>\NETLOGONDX-Report.csv. Overall = HEALTHY when every signal is OK. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 14)

How to run: Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Report.ps1 Optional: -OutputDir C:\ProgramData\NETLOGONDX -EventLookbackDays 14 At scale: run under SYSTEM via Intune Proactive Remediations (as an additional detection package) and harvest <OutputDir>\NETLOGONDX-Report.csv from each device. EXPECTED RESULTS A formatted summary printed to the console and written to <OutputDir>\NETLOGONDX-Report.txt, plus a single-row <OutputDir>\NETLOGONDX-Report.csv. Overall = HEALTHY when every signal is OK. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([string]$OutputDir = "$env:ProgramData\NETLOGONDX", [int]$EventLookbackDays = 14)

NETLOGONDX-Validation.ps1

Purpose: Runs LOCALLY and returns a clear PASS/FAIL verdict against the four things that must be true for a domain-joined Windows device to authenticate without Netlogon 5719: 1. the machine secure channel is healthy (Test-ComputerSecureChannel), 2. Windows Time is in sync with a domain controller (Kerberos skew < 5 minutes), 3. DNS resolves the domain DC locator SRV records (_ldap._tcp.dc._msdcs.<domain>), 4. IPv6 is NOT invalidly disabled (DisabledComponents is 0x0, or a supported value). Designed to double as an Intune Proactive Remediations DETECTION script: it exits 0 when the device is healthy (compliant) and exits 1 when any check fails (triggers the paired NETLOGONDX-Remediation.ps1). Read-only — it never changes anything. HOW TO RUN Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Validation.ps1 As an Intune Proactive Remediations detection script: Reports (Endpoint analytics) > Proactive remediations > Create script package Detection script = NETLOGONDX-Validation.ps1 · Remediation script = NETLOGONDX-Remediation.ps1 Run this script using the logged-on credentials = No (runs as SYSTEM); 64-bit PowerShell = Yes. EXPECTED RESULTS A PASS/FAIL table and an overall verdict. Exit code 0 = HEALTHY, exit code 1 = ISSUES FOUND. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([int]$MaxTimeSkewSeconds = 300)

How to run: Local, elevated: powershell -ExecutionPolicy Bypass -File .\NETLOGONDX-Validation.ps1 As an Intune Proactive Remediations detection script: Reports (Endpoint analytics) > Proactive remediations > Create script package Detection script = NETLOGONDX-Validation.ps1 · Remediation script = NETLOGONDX-Remediation.ps1 Run this script using the logged-on credentials = No (runs as

SYSTEM); 64-bit PowerShell = Yes. EXPECTED RESULTS A PASS/FAIL table and an overall verdict. Exit code 0 = HEALTHY, exit code 1 = ISSUES FOUND. .NOTES White-label — the repeatable way. Read-only diagnostics. > [CmdletBinding()] param([int]$MaxTimeSkewSeconds = 300)